t2c

October 27, 2026

[3]:
```
"""
Maximum-entropy probabilities for two distinguishable three-level atoms

Each atom has levels 0, eps, 2*eps, so the pair has 9 microstates

With the energy constraint <E> = 3*eps/2, Eliminating Z:
    (2x + 6x^2 + 6x^3 + 4x^4)/Z = 3/2
    5x^4 + 6x^3 + 3x^2 - 2x - 3 = 0,
    (5x^2 + x - 3)(x^2 + x + 1) = 0
But the second factor has no real roots, so we only need to solve the first one

However, knowing each atom is independent, total energy is the sum of two␣
 ↪independent energies, resulting in:
    5x^2 + x - 3 = 0

Thus we solve here in two ways:
    - solve the quadratic numerically (bisection)
    - use the analytic root x = (sqrt(61) - 1)/10
"""

import math
from scipy.optimize import brentq

# total energy spectrum: E values and degeneracies
energies = [0, 1, 2, 3, 4]
degeneracies = [1, 2, 3, 2, 1]

# microstate labels
states = ["(0,0)",
          "(0,eps)", "(eps,0)",
          "(0,2eps)", "(2eps,0)", "(eps,eps)",
          "(eps,2eps)", "(2eps,eps)",
          "(2eps,2eps)"]


def probabilities(x):
    """Nine canonical probabilities f_j from x = exp(-beta*eps)."""
```

```
    Z = sum(g * x**E for E, g in zip(energies, degeneracies))
    f = []
    for E, g in zip(energies, degeneracies):
        f.extend([x**E / Z] * g)
    return f, Z


def report(x, label):
    f, Z = probabilities(x)
    beta_eps = -math.log(x)
    mean_E = sum(g * E * x**E for E, g in zip(energies, degeneracies)) / Z
    print(f"--- {label}")
    print(f"x = e^(-beta*eps) = {x:.12f}")
    print(f"Z                 = {Z:.12f}")
    print(f"beta*eps = {beta_eps:.8f}   ->   k_B*T = {1/beta_eps:.6f} eps\n")
    for name, fj in zip(states, f):
        print(f"  {name:<12s}  f = {fj:.6f}")
    print(f"check: sum f_j = {sum(f):.10f}")
    print()
    return f


# numerical root
def h(t):
    return 5.0 * t * t + t - 3.0

x_numerical = brentq(h, 0.0, 1.0)
f_num = report(x_numerical, "numerical root")

# analytic root
x_analytic = (math.sqrt(61.0) - 1.0) / 10.0
f_ana = report(x_analytic, "analytic root")

# comparison
print("--- comparison")
print(f"|x_numerical - x_analytic| = {abs(x_numerical - x_analytic):.2e}")
print(f"max |f_numerical - f_analytic| = {max(abs(a - b) for a, b in zip(f_num,␣
 ↪f_ana)):.2e}")
```

```
--- numerical root
x = e^(-beta*eps) = 0.681024967591
Z                 = 4.600252721181
beta*eps = 0.38415631   ->   k_B*T = 2.603107 eps

  (0,0)         f = 0.217379
  (0,eps)       f = 0.148041
  (eps,0)       f = 0.148041
  (0,2eps)      f = 0.100819
```

```
  (2eps,0)      f = 0.100819
  (eps,eps)     f = 0.100819
  (eps,2eps)    f = 0.068661
  (2eps,eps)    f = 0.068661
  (2eps,2eps)   f = 0.046760
check: sum f_j = 1.0000000000

--- analytic root
x = e^(-beta*eps) = 0.681024967591
Z                 = 4.600252721180
beta*eps = 0.38415631   ->   k_B*T = 2.603107 eps

  (0,0)         f = 0.217379
  (0,eps)       f = 0.148041
  (eps,0)       f = 0.148041
  (0,2eps)      f = 0.100819
  (2eps,0)      f = 0.100819
  (eps,eps)     f = 0.100819
  (eps,2eps)    f = 0.068661
  (2eps,eps)    f = 0.068661
  (2eps,2eps)   f = 0.046760
check: sum f_j = 1.0000000000

--- comparison
|x_numerical - x_analytic| = 1.33e-15
max |f_numerical - f_analytic| = 6.38e-16
```